

Análisis Comparativo Crítico Entre Estilos De Arquitectura Distribuida: Microservicios Vs. Monolito Modular

Iván Andrés Villamarín Cuenca
Facultad de Ciencias de la Computación y Diseño Digital
Universidad Técnica Estatal de Quevedo
Repositorio del informe

<https://github.com/ivillamarinc/TA-IND-01-Informe-Acad-mico-Individual-An-lisis-Comparativo-de-Arquitecturas-Distribuidas>

Resumen—Las arquitecturas distribuidas han adquirido una gran importancia en el desarrollo de aplicaciones modernas debido a la necesidad de crear sistemas más flexibles, mantenibles y preparados para el crecimiento. El presente trabajo realiza un análisis comparativo entre las arquitecturas de microservicios y monolito modular, examinando sus características, ventajas, limitaciones y escenarios de aplicación a partir de la revisión de literatura científica reciente. Como caso de estudio, se analizó el Sistema de Control de Laboratorios Informáticos, actualmente implementado mediante una arquitectura monolítica modular. Los resultados muestran que los microservicios ofrecen mayores capacidades de escalabilidad y despliegue independiente, mientras que el monolito modular presenta una menor complejidad operativa y una administración más sencilla. Se concluye que la arquitectura monolítica modular constituye la alternativa más adecuada para las necesidades actuales del sistema; sin embargo, su organización en módulos bien definidos permite considerar una migración gradual hacia microservicios en escenarios de crecimiento futuro y mayores requerimientos de integración y escalabilidad.

Index Terms—arquitecturas distribuidas, microservicios, monolito modular, escalabilidad, migración arquitectónica, sistema de control de laboratorios informáticos.

ABSTRACT

Distributed architectures have become increasingly important in modern software development due to the need for systems that are flexible, maintainable, and capable of supporting future growth. This paper presents a comparative analysis between microservices and modular monolith architectures by examining their characteristics, advantages, limitations, and application scenarios based on recent scientific literature. As a case study, the Laboratory Information Management System, currently implemented as a modular monolith, was analyzed. The findings indicate that microservices provide greater scalability and independent deployment capabilities, whereas the modular monolith offers lower operational complexity and simpler system management. It is concluded that the modular monolith architecture is the most appropriate solution for the current requirements of the system. However, its well-defined modular structure also enables a gradual transition toward

microservices if future demands require higher scalability, integration, and system evolution.

KEYWORDS

distributed architectures, microservices, modular monolith, scalability, architectural migration, laboratory information management system.

I. INTRODUCCIÓN

Las arquitecturas distribuidas han evolucionado para responder a las necesidades de escalabilidad, disponibilidad y mantenimiento de las aplicaciones modernas. Entre los enfoques más utilizados se encuentran los microservicios y el monolito modular, los cuales presentan características, ventajas y desafíos diferentes en el desarrollo de sistemas de software. La selección de una arquitectura adecuada constituye una decisión importante, ya que influye directamente en el rendimiento, la complejidad operativa y la capacidad de evolución de una aplicación.

El presente trabajo tiene como objetivo realizar un análisis comparativo entre ambas arquitecturas y evaluar su aplicación en un Sistema de Control de Laboratorios Informáticos. Para ello, se empleó una metodología basada en la revisión y análisis de literatura científica reciente. El estudio se limita al análisis conceptual y arquitectónico de ambos enfoques. Como resultado, se busca identificar la alternativa más adecuada para el contexto del proyecto, aportando criterios que puedan servir como referencia para próximas decisiones de diseño y evolución del sistema.

I-A. Fundamentos de arquitecturas distribuidas

Las arquitecturas distribuidas están conformadas por múltiples componentes de software que cooperan a través de una red para proporcionar servicios de manera coordinada. Este enfoque permite distribuir recursos, mejorar la escalabilidad y aumentar la disponibilidad de las aplicaciones modernas. Sin embargo, la distribución de componentes también introduce desafíos relacionados con la comunicación, sincronización y administración de recursos compartidos. Estas características

han convertido a las arquitecturas distribuidas en la base de tecnologías actuales como la computación en la nube, los microservicios y los sistemas de gran escala [1], [2].

Uno de los principios más importantes en sistemas distribuidos es la consistencia de datos, que se refiere a la capacidad de mantener información coherente entre diferentes nodos o réplicas. La replicación mejora la disponibilidad y la tolerancia a fallos, pero incrementa la complejidad para garantizar que todos los nodos observen estados compatibles de la información. Por esta razón, diversos modelos de consistencia han sido desarrollados para equilibrar los requerimientos de rendimiento, disponibilidad y precisión de los datos en entornos distribuidos [3], [4].

Otro fundamento ampliamente aceptado es el teorema CAP, el cual establece que, ante una partición de red, un sistema distribuido no puede garantizar simultáneamente consistencia y disponibilidad. En consecuencia, los arquitectos de software deben decidir qué propiedad priorizar según las necesidades del sistema. Este principio resulta especialmente relevante en arquitecturas modernas basadas en servicios distribuidos, donde las decisiones de diseño influyen directamente en el comportamiento del sistema frente a fallos de comunicación o interrupciones de red [5], [6].

La tolerancia a fallos constituye otro aspecto esencial de las arquitecturas distribuidas. Esta propiedad permite que un sistema continúe operando aun cuando algunos de sus componentes experimenten errores o interrupciones. Para ello se emplean mecanismos como redundancia, replicación, monitoreo y recuperación automática, con el objetivo de minimizar el impacto de las fallas y garantizar la continuidad operativa de los servicios. Estas estrategias son particularmente importantes en aplicaciones que requieren altos niveles de disponibilidad y confiabilidad [7], [8].

Los fundamentos descritos sirven como base para comprender estilos arquitectónicos como el monolito modular y los microservicios. Ambos enfoques buscan mejorar la organización y mantenibilidad del software, pero difieren en la manera en que distribuyen responsabilidades, gestionan la comunicación entre componentes y enfrentan los desafíos inherentes a los sistemas distribuidos. Por ello, comprender estos principios resulta indispensable para evaluar las ventajas y limitaciones de cada arquitectura [9], [10].

II. ARQUITECTURA DE MICROSERVICIOS

La arquitectura de microservicios es un estilo de diseño de software que estructura una aplicación como un conjunto de servicios independientes que colaboran entre sí para cumplir funciones específicas del negocio. Cada servicio posee responsabilidades claramente definidas y puede desarrollarse, desplegarse y mantenerse de forma autónoma. Este enfoque surgió como una alternativa a las limitaciones de las arquitecturas monolíticas tradicionales, especialmente en aplicaciones que requieren alta escalabilidad y evolución continua [9], [10].

Uno de los principios fundamentales de los microservicios es la descentralización de responsabilidades. Cada servicio administra su propia lógica de negocio y, en muchos casos, su

propio almacenamiento de datos. Esta independencia favorece la modularidad y permite que diferentes equipos trabajen simultáneamente sobre distintos componentes del sistema. Asimismo, facilita la actualización de funcionalidades sin afectar el funcionamiento completo de la aplicación [11], [12].

La comunicación entre microservicios constituye un aspecto esencial para el correcto funcionamiento de la arquitectura. Los servicios intercambian información mediante mecanismos síncronos, como APIs REST, o asíncronos, mediante colas de mensajes y eventos. La selección de un patrón de comunicación adecuado influye directamente en el rendimiento, la resiliencia y la capacidad de escalamiento del sistema distribuido [13], [14].

Entre las principales ventajas de esta arquitectura se encuentran la escalabilidad independiente de los servicios, el aislamiento de fallos y la posibilidad de realizar despliegues continuos. Estas características permiten que aplicaciones complejas mantengan altos niveles de disponibilidad y respondan eficientemente a cambios en la demanda. Sin embargo, la distribución de componentes también incrementa la complejidad relacionada con monitoreo, seguridad, pruebas y gestión de infraestructura [15], [16].

A pesar de sus beneficios, la adopción de microservicios no resulta adecuada para todos los proyectos. Su implementación requiere experiencia en sistemas distribuidos, automatización, observabilidad y gestión de comunicaciones entre servicios. Por esta razón, diversos estudios coinciden en que la decisión de migrar desde una arquitectura monolítica debe estar respaldada por necesidades reales de escalabilidad, crecimiento organizacional y evolución tecnológica del sistema [10], [17].

III. ARQUITECTURA MONOLÍTICA MODULAR

La arquitectura monolítica modular es un estilo de diseño de software en el que la aplicación se despliega como una única unidad ejecutable, pero se encuentra organizada internamente en módulos con responsabilidades claramente definidas. Cada módulo encapsula su lógica de negocio y mantiene límites funcionales que favorecen la separación de responsabilidades y reducen el acoplamiento entre componentes. En los últimos años, esta arquitectura ha despertado un renovado interés debido a su capacidad para combinar la simplicidad operativa de los monolitos con los beneficios de la modularidad [18], [19].

Uno de los principios fundamentales del monolito modular es la organización del sistema en módulos independientes desde el punto de vista lógico, aunque todos formen parte de la misma aplicación. Esta estructura facilita el mantenimiento del código, mejora la comprensión del sistema y permite que diferentes equipos trabajen sobre distintos módulos sin introducir una complejidad operativa elevada. Asimismo, favorece la evolución gradual del sistema y puede servir como una etapa previa antes de adoptar una arquitectura de microservicios [18], [20].

La comunicación entre módulos se realiza generalmente mediante llamadas internas, interfaces bien definidas o mecanismos basados en eventos, evitando la necesidad de comu-

nicaciones remotas a través de la red. Como consecuencia, se reducen problemas asociados con latencia, serialización de datos y coordinación entre servicios distribuidos. Esta característica permite mantener un rendimiento predecible y simplifica la implementación de transacciones que involucran múltiples módulos del sistema [21], [20].

Entre las principales ventajas de esta arquitectura se encuentran la simplicidad de despliegue, la menor complejidad de infraestructura y la facilidad para depurar y monitorear la aplicación. Al mantenerse como una única unidad desplegable, se reducen los costos operativos y administrativos asociados con la orquestación de múltiples servicios. Además, la modularidad interna favorece la mantenibilidad y disminuye el riesgo de que el sistema evolucione hacia un código altamente acoplado y difícil de gestionar [18], [21].

A pesar de sus beneficios, la arquitectura monolítica modular también presenta limitaciones. La aplicación continúa siendo desplegada como una única unidad, por lo que la escalabilidad independiente de cada módulo es restringida y una falla crítica puede afectar al sistema completo. No obstante, diversos estudios recientes sostienen que esta arquitectura constituye una alternativa adecuada para proyectos de tamaño pequeño y mediano, así como para organizaciones que buscan reducir la complejidad operativa sin renunciar a una adecuada organización interna del software [20], [17].

IV. COMPARACIÓN ENTRE AMBAS ARQUITECTURAS

Las arquitecturas de microservicios y monolito modular comparten el objetivo de mejorar la organización y mantenibilidad del software mediante la separación de responsabilidades en componentes funcionales. Sin embargo, difieren significativamente en la forma de implementar dicha modularidad. Mientras el monolito modular mantiene todos sus módulos dentro de una única aplicación desplegable, los microservicios distribuyen las funcionalidades en servicios independientes que pueden desarrollarse, desplegarse y ejecutarse de manera autónoma. Esta diferencia tiene implicaciones directas en aspectos relacionados con complejidad operativa, rendimiento, escalabilidad y mantenimiento del sistema [9], [18].

En términos de escalabilidad, la arquitectura de microservicios ofrece una mayor flexibilidad, ya que permite escalar únicamente los servicios que experimentan una mayor carga de trabajo. Por el contrario, el monolito modular generalmente requiere escalar la aplicación completa, incluso cuando solo algunos módulos demandan mayores recursos. No obstante, diversos estudios señalan que la mayoría de las aplicaciones empresariales no presentan niveles de demanda que justifiquen la complejidad adicional asociada con una infraestructura de microservicios, por lo que un monolito modular puede satisfacer adecuadamente los requerimientos de muchos sistemas de tamaño pequeño y mediano [20], [10].

Respecto al rendimiento y la comunicación interna, el monolito modular presenta ciertas ventajas al ejecutar todos sus módulos dentro del mismo proceso. La comunicación mediante llamadas internas reduce la latencia y elimina la sobrecarga generada por la serialización de datos y las comunicaciones de

red. En contraste, los microservicios dependen de mecanismos de comunicación distribuida, como APIs REST o mensajería basada en eventos, los cuales proporcionan flexibilidad arquitectónica, pero introducen desafíos relacionados con tiempos de respuesta, monitoreo y coordinación entre servicios [13], [21].

Desde la perspectiva de la mantenibilidad y evolución del software, ambas arquitecturas ofrecen beneficios importantes, aunque bajo condiciones diferentes. Los microservicios favorecen el desarrollo independiente de componentes y facilitan la incorporación de nuevas funcionalidades sin afectar la totalidad del sistema. Sin embargo, también incrementan la complejidad asociada con pruebas, observabilidad, seguridad y administración de múltiples servicios. Por su parte, el monolito modular mantiene una estructura organizada y desacoplada, pero conserva la simplicidad de desplegar y administrar una única aplicación, reduciendo considerablemente la carga operativa del sistema [11], [19].

Otro aspecto diferenciador corresponde a la tolerancia a fallos y la disponibilidad. En los microservicios, el aislamiento entre servicios puede limitar la propagación de determinados errores y facilitar mecanismos de recuperación independiente. Sin embargo, la distribución de componentes incrementa la complejidad de garantizar consistencia, coordinación y resiliencia ante fallos de comunicación. En contraste, el monolito modular simplifica la gestión transaccional y la consistencia de datos al ejecutarse dentro de un único proceso, aunque determinadas fallas críticas pueden afectar a toda la aplicación al existir un único artefacto desplegable [1], [2].

Finalmente, la evidencia científica reciente coincide en que ninguna de las dos arquitecturas puede considerarse universalmente superior. La selección de una alternativa debe responder a las necesidades del dominio de aplicación, los recursos técnicos disponibles, el tamaño del equipo de desarrollo y las perspectivas de crecimiento del sistema. En proyectos con requisitos moderados de escalabilidad y equipos reducidos, el monolito modular constituye una solución eficiente y de menor complejidad operativa. Por el contrario, en aplicaciones de gran escala, con altos niveles de crecimiento y requerimientos de despliegue independiente, la arquitectura de microservicios puede ofrecer ventajas significativas a largo plazo [9], [19].

La evidencia científica analizada indica que la elección entre una arquitectura de microservicios y un monolito modular representa un proceso de compensación (trade-off) entre flexibilidad y simplicidad operativa. Los microservicios proporcionan ventajas significativas en términos de escalabilidad independiente, autonomía de despliegue y capacidad de evolución de componentes específicos; sin embargo, estas ventajas se obtienen a costa de una mayor complejidad asociada con la comunicación distribuida, la observabilidad, la coordinación de datos y la administración de infraestructura [9], [10], [13]. Por el contrario, el monolito modular conserva una estructura organizada y desacoplada internamente, pero reduce considerablemente la complejidad de operación al ejecutarse como una única aplicación desplegable, favoreciendo la mantenibilidad y la gestión de transacciones [18], [21], [19].

Cuadro I
COMPARACIÓN ENTRE LAS DOS ARQUITECTURAS ESCOGIDAS

Criterio	Microservicios	Monolito Modular	Referencias de respaldo
Unidad de despliegue	Cada servicio se despliega de forma independiente.	Toda la aplicación se despliega como un único artefacto.	[9], [18], [1]
Escalabilidad	Permite escalar únicamente los servicios con mayor demanda.	Generalmente requiere escalar toda la aplicación.	[20], [10]
Comunicación	Utiliza APIs REST, mensajería o eventos distribuidos.	Emplea llamadas internas entre módulos.	[13], [21]
Rendimiento	Puede verse afectado por la latencia y la sobrecarga de red.	Presenta menor latencia al ejecutarse en un único proceso.	[13], [21]
Complejidad operativa	Requiere herramientas de orquestación, monitoreo y gestión distribuida.	Posee una infraestructura más simple y menor sobrecarga administrativa.	[18], [11], [19]
Mantenibilidad	Facilita el desarrollo y despliegue independiente de servicios.	Facilita la organización del código manteniendo un único entorno de ejecución.	[9], [18], [11]
Consistencia de datos	La coordinación transaccional es más compleja al existir servicios distribuidos.	La gestión transaccional es más sencilla al compartir el mismo proceso y, frecuentemente, la misma base de datos.	[1], [2]
Tolerancia a fallos	El aislamiento de servicios puede limitar la propagación de errores.	Una falla crítica puede afectar a toda la aplicación.	[1], [2]
Costos de infraestructura	Generalmente mayores debido a la administración de múltiples servicios.	Menores por la existencia de una única aplicación desplegable.	[18], [20], [19]
Escenarios recomendados	Sistemas de gran escala, equipos numerosos y requisitos elevados de escalabilidad y evolución.	Sistemas pequeños y medianos, equipos reducidos y necesidades moderadas de crecimiento.	[9], [20], [10], [19]

Asimismo, la literatura reciente coincide en que la adopción de microservicios no debe considerarse un objetivo por sí mismo ni una evolución obligatoria de todos los sistemas de software. Diversos estudios muestran que numerosos proyectos empresariales mantienen requisitos de escalabilidad moderados y equipos de desarrollo reducidos, condiciones en las cuales la complejidad adicional de una arquitectura distribuida puede superar los beneficios obtenidos [20], [10], [19]. En estos escenarios, el monolito modular emerge como una alternativa arquitectónica equilibrada que permite aplicar principios de modularidad y separación de responsabilidades sin incurrir en los costos operativos propios de los microservicios [18], [21].

Finalmente, puede afirmarse que ninguna de las dos arquitecturas es universalmente superior. La decisión arquitectónica debe responder al contexto del proyecto, considerando factores como el tamaño del sistema, el ritmo esperado de crecimiento, la disponibilidad de recursos técnicos, los requisitos de disponibilidad y la capacidad del equipo para administrar la complejidad operacional. En consecuencia, la selección de una arquitectura debe fundamentarse en un análisis de necesidades y restricciones del dominio de aplicación, priorizando el equilibrio entre mantenibilidad, rendimiento y escalabilidad a largo plazo [9], [18], [20], [19].

En definitiva, la elección entre microservicios y monolito modular no debe basarse en tendencias tecnológicas, sino en la capacidad de cada arquitectura para responder de manera eficiente a las necesidades funcionales, operativas y de crecimiento del dominio de aplicación considerado.

V. APLICACIÓN AL PROYECTO INTEGRADOR PROPIO

El proyecto integrador desarrollado corresponde a un Sistema de Control de Laboratorios Informáticos orientado a la gestión de recursos, reservas y seguimiento de las actividades realizadas dentro de los laboratorios de la institución. Actualmente, el sistema se encuentra implementado bajo una arquitectura monolítica modular, en la cual las funcionalidades se encuentran organizadas en módulos independientes desde el punto de vista lógico, pero desplegadas como una única aplicación. Entre los principales módulos se encuentran Inventario, Horarios y Reservas, Asistencia, Reportes Técnicos, Administración, Historial y Reportes, además del mecanismo de autenticación y control de acceso mediante usuarios y roles.

Desde una perspectiva arquitectónica, el monolito modular ha demostrado ser adecuado para las necesidades actuales del sistema. La separación de responsabilidades entre módulos facilita el mantenimiento del código, permite la evolución gradual de las funcionalidades y simplifica las tareas de despliegue y administración al existir una única aplicación y una infraestructura reducida. Asimismo, la comunicación interna mediante llamadas directas entre módulos disminuye la latencia y simplifica la implementación de operaciones que requieren intercambiar información entre inventario, reservas, asistencia y reportes.

Dando un ejemplo, cuando un docente realiza una reserva de laboratorio, el sistema debe verificar la disponibilidad de horarios, consultar la información del laboratorio, validar los permisos del usuario autenticado y posteriormente registrar la reserva en el historial correspondiente. De igual manera, el registro de asistencia requiere interactuar con los módulos de autenticación, horarios y generación de reportes. Estas operaciones presentan una fuerte dependencia funcional y

requieren un intercambio constante de información, situación en la que una arquitectura monolítica modular proporciona un flujo de ejecución más sencillo y una menor complejidad operativa.

La adopción de una arquitectura de microservicios permitiría escalar de manera independiente determinados módulos, como el de Asistencia o Historial y Reportes, en escenarios donde la cantidad de usuarios, laboratorios o consultas creciera significativamente. Además, facilitaría la implementación de despliegues independientes y permitiría que equipos de desarrollo diferentes trabajen sobre servicios específicos de manera autónoma. Sin embargo, dicha migración también introduciría nuevos desafíos relacionados con la comunicación distribuida, la coordinación de datos, la observabilidad, la seguridad entre servicios y la administración de una infraestructura considerablemente más compleja.

Considerando el contexto actual del proyecto, si se considerara una migración progresiva hacia una arquitectura de microservicios, el módulo más recomendable para iniciar este proceso sería Historial y Reportes. Este módulo presenta un menor acoplamiento funcional respecto a los procesos transaccionales del sistema y sus operaciones se centran principalmente en la consulta, consolidación y exportación de información. De la misma manera, el crecimiento en la generación de reportes institucionales podría demandar mayores recursos de procesamiento y almacenamiento, justificando la necesidad de un escalamiento independiente. Debido a estas características, la migración inicial de este módulo permitiría introducir gradualmente los principios de una arquitectura distribuida, reducir los riesgos operativos y adquirir experiencia en la gestión de servicios independientes antes de abordar módulos más complejos como lo son la autenticación, reservas o asistencia.

V-A. Arquitectura propuesta para implementar en SCLI

En esta arquitectura, los usuarios interactúan inicialmente con un API Gateway, el cual centraliza las solicitudes y las redirige hacia los microservicios correspondientes. Cada servicio administra su propia base de datos y se responsabiliza únicamente de un dominio específico del sistema. Por ejemplo, el microservicio de Asistencia consulta al servicio de Horarios y Reservas para validar la disponibilidad del laboratorio, mientras que el servicio de Historial y Reportes consume información proveniente de varios servicios para generar estadísticas e informes institucionales. Esta organización permite escalar y desplegar cada servicio de manera independiente; sin embargo, también introduce desafíos relacionados con la comunicación distribuida, la consistencia de datos y el monitoreo de múltiples servicios como se puede ver en la Ilustración 1: Arquitectura propuesta (ver Anexos).

VI. DISCUSIÓN CRÍTICA

El análisis realizado muestra que no existe una arquitectura de software que sea la mejor en todos los casos. La elección depende de las necesidades actuales y futuras del sistema. Los microservicios ofrecen ventajas como la escalabilidad,

la flexibilidad y la posibilidad de actualizar componentes de manera independiente, aunque también incrementan la complejidad de desarrollo y operación del sistema [9], [10].

En el Sistema de Control de Laboratorios Informáticos, la arquitectura monolítica modular continúa siendo adecuada para las necesidades actuales de la institución. Sin embargo, la organización del sistema en módulos bien definidos facilita una posible transición hacia microservicios si en el futuro aumenta el número de usuarios, se requiere integrar nuevas funcionalidades o se necesita escalar determinados procesos de manera independiente [18], [20].

Por lo tanto, la decisión más conveniente es mantener la arquitectura monolítica modular en el corto plazo y considerar una migración gradual hacia microservicios únicamente cuando las necesidades de crecimiento y evolución del sistema justifiquen asumir la mayor complejidad de una arquitectura distribuida [10], [18].

VII. CONCLUSIONES

El análisis realizado permitió comprender que la elección de una arquitectura de software depende de las necesidades y características de cada sistema. Aunque los microservicios ofrecen mayor flexibilidad y capacidad de crecimiento, también requieren una administración más compleja. En el caso del Sistema de Control de Laboratorios Informáticos, la arquitectura monolítica modular resulta más adecuada para las necesidades actuales, ya que permite mantener una organización clara de los módulos y una gestión más sencilla. Sin embargo, su estructura modular deja abierta la posibilidad de evolucionar gradualmente hacia microservicios si en el futuro aumentan los usuarios, las funcionalidades o los requerimientos de integración con otros sistemas.

DECLARACIÓN DE USO DE INTELIGENCIA ARTIFICIAL

Para la elaboración del presente informe se utilizaron herramientas de Inteligencia Artificial (IA) como apoyo en la organización de ideas, corregir ambigüedades en la redacción, revisión gramatical y generación de sugerencias para la estructura del documento. De la misma manera, la IA se empleó como herramienta de apoyo para sintetizar conceptos y orientar la búsqueda de literatura científica relacionada con las arquitecturas distribuidas.

La selección de las fuentes bibliográficas, la revisión de su pertinencia y la verificación de las referencias fueron realizadas manualmente por el autor. Cada cita utilizada en el informe fue comprobada de manera individual mediante la revisión de sus metadatos, DOI y contenido académico correspondiente. Para la consulta de los artículos científicos se utilizaron repositorios académicos y plataformas de acceso a publicaciones científicas, entre ellas ResearchGate y Sci-Hub otros servicios de consulta disponibles para fines de revisión bibliográfica.

La interpretación de la información, el análisis comparativo entre las arquitecturas estudiadas, la aplicación al Sistema de Control de Laboratorios Informáticos y las conclusiones presentadas son responsabilidad exclusiva del autor del informe.

REFERENCIAS

- [1] E. A. Lee, R. Akella, S. Bateni, S. Lin, M. Lohstroh, and C. Menard, "Consistency vs. Availability in Distributed Cyber-Physical Systems," *ACM Transactions on Embedded Computing Systems*, vol. 22, no. 5s, Sep. 2023, doi: 10.1145/3609119.
- [2] A. A. Hussein et al., "State of Art Survey for Fault Tolerance Feasibility in Distributed Systems," *Asian Journal of Research in Computer Science*, pp. 19–34, Sep. 2021, doi: 10.9734/AJRCOS/2021/V11I1430268.
- [3] R. Zennou, R. Biswas, A. Bouajjani, C. Enea, and M. Erradi, "Checking Causal Consistency of Distributed Databases," *Computing*, vol. 104, no. 10, pp. 2181–2201, Feb. 2021, doi: 10.1007/s00607-021-00911-3.
- [4] V. R. Cadambe and S. Lyu, "CausalEC: A Causally Consistent Data Storage Algorithm based on Cross-Object Erasure Coding," Feb. 2021, Accessed: Jun. 10, 2026. [Online]. Available: <https://arxiv.org/pdf/2102.13310>
- [5] E. A. Lee, S. Bateni, S. Lin, M. Lohstroh, and C. Menard, "Trading Off Consistency and Availability in Tiered Heterogeneous Distributed Systems," *Intelligent Computing*, vol. 2, Feb. 2023, doi: 10.34133/ICOMPUTING.0013.
- [6] H. Nejati, S. Aldin, H. Deldari, M. H. Moattar, and M. R. Ghods, "Consistency models in distributed systems: A survey on definitions, disciplines, challenges and applications," Feb. 2019, Accessed: Jun. 10, 2026. [Online]. Available: <https://arxiv.org/pdf/1902.03305>
- [7] A. A. Hussein et al., "State of Art Survey for Fault Tolerance Feasibility in Distributed Systems," *Asian Journal of Research in Computer Science*, pp. 19–34, Sep. 2021, doi: 10.9734/AJRCOS/2021/V11I1430268.
- [8] S. Liu, "A Survey on Fault-tolerance in Distributed Optimization and Machine Learning," Jun. 2021, Accessed: Jun. 10, 2026. [Online]. Available: <https://arxiv.org/pdf/2106.08545>
- [9] V. Velepucha and P. Flores, "A Survey on Microservices Architecture: Principles, Patterns and Migration Challenges," *IEEE Access*, vol. 11, pp. 88339–88358, 2023, doi: 10.1109/ACCESS.2023.3305687.
- [10] H. M. Ayas, P. Leitner, R. Hebig, X. Peng, B. Hamdy, and M. Ayas, "An empirical study of the systemic and technical migration towards microservices," *Empirical Software Engineering*, vol. 28, no. 4, pp. 85–, May 2023, doi: 10.1007/S10664-023-10308-9.
- [11] M. Waseem, P. Liang, M. Shahin, A. Di Salle, and G. Márquez, "Design, monitoring, and testing of microservices systems: The practitioners' perspective," *Journal of Systems and Software*, vol. 182, p. 111061, Dec. 2021, doi: 10.1016/J.JSS.2021.111061.
- [12] D. Taibi, V. Lenarduzzi, and C. Pahl, "Processes, Motivations, and Issues for Migrating to Microservices Architectures: An Empirical Investigation," *IEEE Cloud Computing*, vol. 4, no. 5, pp. 22–32, Sep. 2017, doi: 10.1109/MCC.2017.4250931.
- [13] I. Karabey Aksakalli, T. Çelik, A. B. Can, and B. Tekinerdoğan, "Deployment and communication patterns in microservice architectures: A systematic literature review," *Journal of Systems and Software*, vol. 180, p. 111014, Oct. 2021, doi: 10.1016/J.JSS.2021.111014.
- [14] J. Soldani, D. A. Tamburri, and W. J. Van Den Heuvel, "The pains and gains of microservices: A Systematic grey literature review," *Journal of Systems and Software*, vol. 146, pp. 215–232, Dec. 2018, doi: 10.1016/J.JSS.2018.09.082.
- [15] N. Mateus-Coelho, M. Cruz-Cunha, and L. G. Ferreira, "Security in Microservices Architectures," *Procedia Comput. Sci.*, vol. 181, pp. 1225–1236, Jan. 2021, doi: 10.1016/J.PROCS.2021.01.320.
- [16] H. Siddiqui, F. Khendek, and M. Toeroe, "Microservices based architectures for IoT systems - State-of-the-art review," *Internet of Things*, vol. 23, p. 100854, Oct. 2023, doi: 10.1016/J.IOT.2023.100854.
- [17] A. Martínez Saucedo, G. Rodríguez, F. Gomes Rocha, and R. P. dos Santos, "Migration of monolithic systems to microservices: A systematic mapping study," *Inf. Softw. Technol.*, vol. 177, p. 107590, Jan. 2025, doi: 10.1016/J.INFSOF.2024.107590.
- [18] L. F. Al-Qora'n and A. Al-Said Ahmad, "Modular Monolith Architecture in Cloud Environments: A Systematic Literature Review," *Future Internet*, vol. 17, no. 11, p. 496, Oct. 2025, doi: 10.3390/FI17110496.
- [19] R. Su and X. Li, "Modular Monolith: Is This the Trend in Software Architecture?," in *Proceedings - 2024 IEEE/ACM International Workshop New Trends in Software Architecture, SATrends 2024*, pp. 10–13, Aug. 2024, doi: 10.1145/3643657.3643911.
- [20] D. Faustino, N. Gonçalves, M. Portela, and A. Rito Silva, "Stepwise migration of a monolith to a microservice architecture: Performance and migration effort evaluation," *Performance Evaluation*, vol. 164, p. 102411, May 2024, doi: 10.1016/J.PEVA.2024.102411.
- [21] V. V. Horshkov, N. M. Lishchyna, and V. A. Sychuk, "Modulnyi monolit: arkhitekturnyi pidkhd dlia pobudovy vysokodostupnoi systemy upravlinnia zariadnymi stantsiiamy" [Modular monolith: an architectural approach for building a high-availability charging station management system], *Computer-Integrated Technologies: Education, Science, Production*, no. 57, pp. 31–42, Feb. 2024, doi: 10.36910/6775-2524-0560-2024-57-05.

Cuadro II
GLOSARIO DE PALABRAS CLAVE

Término	Definición concisa	Referencias
Sistema distribuido	Conjunto de computadoras que trabajan de manera coordinada y se presentan al usuario como un solo sistema.	[1], [2]
Consistencia	Propiedad que garantiza que los datos mantengan el mismo estado en los diferentes componentes de un sistema distribuido.	[3], [6]
Disponibilidad	Capacidad de un sistema para permanecer operativo y responder a las solicitudes de los usuarios aun cuando existan fallos parciales.	[1], [5]
Tolerancia a fallos	Capacidad de un sistema para continuar funcionando correctamente ante errores o fallos de algunos de sus componentes.	[2], [8]
Arquitectura de software	Organización de los componentes de un sistema y la forma en que interactúan para satisfacer requisitos funcionales y no funcionales.	[9], [18]
Microservicios	Estilo arquitectónico que divide una aplicación en servicios pequeños e independientes, cada uno encargado de una función específica del negocio.	[9], [16]
Monolito modular	Arquitectura en la que la aplicación se despliega como una sola unidad, pero está organizada internamente en módulos bien definidos e independientes desde el punto de vista lógico.	[18], [19]
Escalabilidad	Capacidad de un sistema para aumentar su capacidad de procesamiento y atender una mayor carga de usuarios o solicitudes.	[9], [20]
Migración arquitectónica	Proceso de transformación de una arquitectura de software hacia otra, por ejemplo, de un monolito a microservicios, buscando mejorar determinados atributos de calidad.	[17], [20]
Despliegue independiente	Capacidad de actualizar o publicar un componente del sistema sin necesidad de modificar o detener el resto de la aplicación.	[10], [13]
Acoplamiento	Grado de dependencia existente entre los componentes de un sistema; un menor acoplamiento facilita el mantenimiento y la evolución del software.	[11], [19]
Observabilidad	Capacidad de supervisar y comprender el comportamiento interno de un sistema mediante métricas, registros y trazas.	[11], [15]
API Gateway	Componente que centraliza las solicitudes de los clientes y las redirige hacia los microservicios correspondientes.	[9], [13]
Comunicación distribuida	Intercambio de información entre servicios que se ejecutan de forma independiente y se comunican a través de la red.	[13], [16]
Modularidad	Principio de diseño que divide un sistema en partes con responsabilidades específicas para facilitar su comprensión, mantenimiento y evolución.	[18], [21]

APÉNDICE
ANEXOS

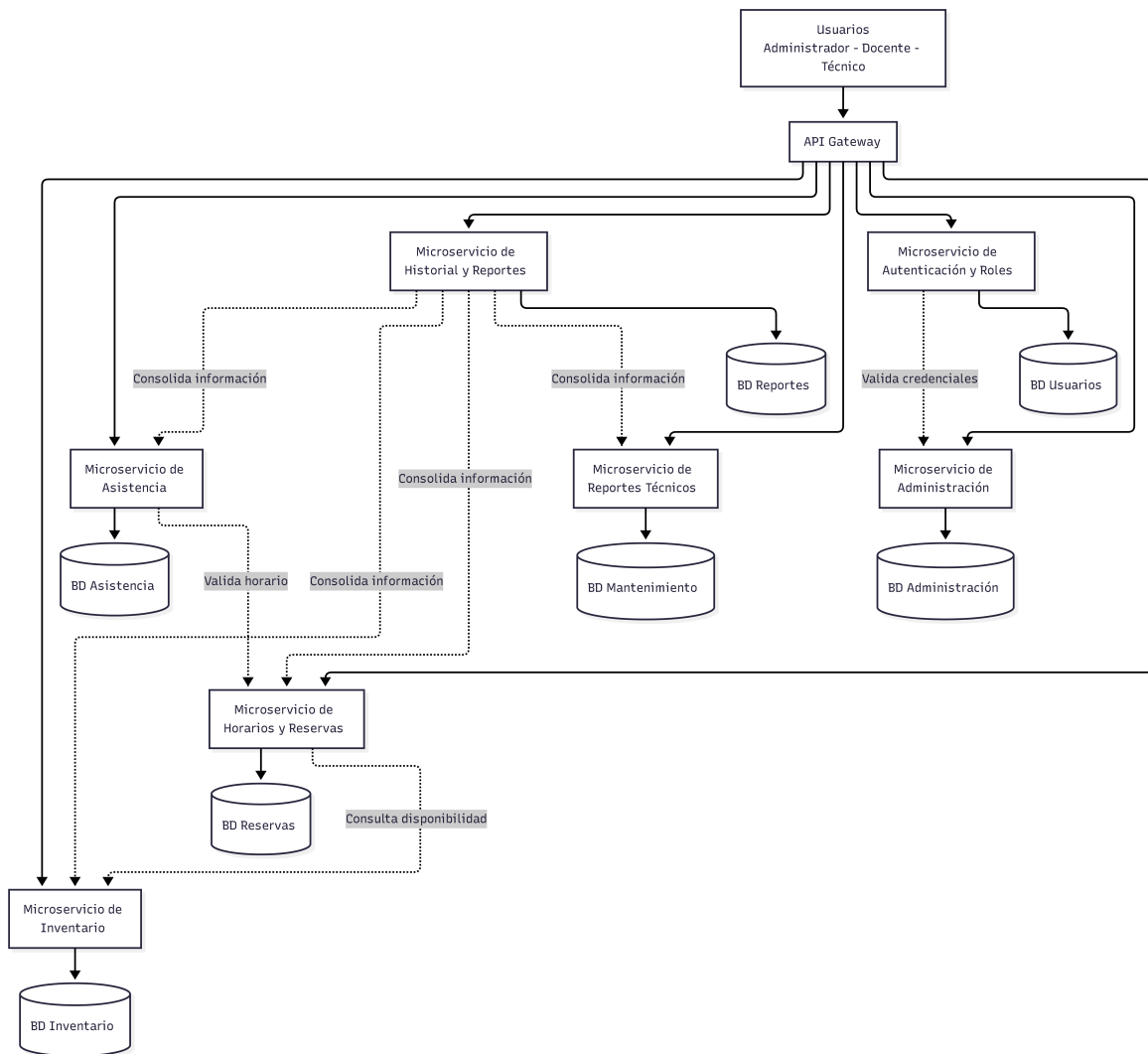


Figura 1. Arquitectura propuesta para el Sistema de Control de Laboratorios Informáticos (SCLI).